

TEAM 5 — MERIDIAN RESIDENCES

ONE EPIC • FOUR FULL-STACK USER STORIES

EPIC — Meridian Residences Resident Portal

Epic Description: Build one integrated resident-facing application for Meridian Residences. Residents can view their lease, track recurring invoices, submit and track maintenance requests, and ask questions about lease and building policies. All four stories share one React frontend, one FastAPI backend/API layer and one PostgreSQL database.

Technology: React + FastAPI + PostgreSQL + pgvector

Team Structure: Four members. Each member owns one complete full-stack story — UI, APIs, backend logic, database work, integration and unit tests. At the end, all stories are integrated and submitted as ONE application.

Member	Story	Ownership
Member 1	View Lease Details	LeaseAgreement + lease UI/API
Member 2	Recurring Invoices	RecurringInvoice + billing UI/API
Member 3	Maintenance	MaintenanceTicket + AI triage
Member 4	AI Assistant	RAG + lease/policy Q&A;

USER STORY 1 — View Lease Details

Owner: Member 1

User Story / Description

As a resident, I want to view my active lease and apartment details so that I can understand my current agreement and important lease dates.

Implementation Scope

- Create the React lease page in the shared resident portal.
- Create FastAPI API(s) to retrieve the resident's active LeaseAgreement.
- Store and retrieve lease information from PostgreSQL.
- Display unit, lease term, monthly rate and renewal date with loading/error handling.

Acceptance Criteria

- A resident can view the active lease and its important details.
- The API returns unit, term, monthly rate and renewal date.
- React consumes the FastAPI API and displays the returned data.
- Invalid or missing lease data produces a clear API/UI error.

Unit Test Cases

Test Case	Expected Result
Valid lease request	Lease data is returned with HTTP 200.
Invalid lease ID	API returns HTTP 404.
Required field validation	Invalid lease data is rejected.
Frontend success	Lease details are displayed correctly.
Frontend error	A clear error message is shown.

USER STORY 2 — View & Generate Recurring Invoices

Owner: Member 2

User Story / Description

As a resident, I want to view my upcoming recurring invoices and payment status so that I can track my payments and identify overdue amounts.

Implementation Scope

- Create the React invoice section in the same resident portal.
- Create FastAPI APIs to list and generate RecurringInvoice records.
- Store invoices against LeaseAgreement in PostgreSQL.
- Handle upcoming, paid and overdue payment status.
- Keep the billing logic ready for the required monthly UiPath workflow.

Acceptance Criteria

- Resident can view billing period, amount and payment status.
- An invoice can be generated for an active lease.
- Generated invoices are stored against the correct lease.
- Unpaid invoices from a previous billing cycle can be identified as overdue.
- The invoice feature uses the same frontend, backend and database as the other stories.

Unit Test Cases

Test Case	Expected Result
List invoices	Correct invoices for the lease are returned.
Generate invoice	A RecurringInvoice record is created.
Invalid/inactive lease	Invoice generation is rejected.
Overdue check	Past-due unpaid invoice is flagged overdue.
Frontend display	Period, amount and status are displayed.

USER STORY 3 — Submit, Track & Triage Maintenance

Owner: Member 3

User Story / Description

As a resident, I want to submit and track maintenance requests so that apartment issues are handled quickly and urgent problems are escalated.

Implementation Scope

- Create the React maintenance form and ticket/status view.
- Create FastAPI APIs to create and retrieve MaintenanceTicket records.
- Store tickets in PostgreSQL.
- Integrate AI triage to determine issue type and priority.
- Assign a vendor queue and escalate urgent issues.

Acceptance Criteria

- Resident can submit unit, issue type and description.
- A valid request creates a MaintenanceTicket and returns its status.
- AI determines issue type and priority.
- The ticket is assigned to the appropriate vendor queue.
- Urgent issues such as water leaks or no heat are automatically escalated.
- Resident can view the current status of submitted tickets.

Unit Test Cases

Test Case	Expected Result
Create valid ticket	MaintenanceTicket is created successfully.
Missing data	Validation error is returned.
Water leak	Ticket is classified urgent and escalated.
Normal request	Correct type/priority and vendor queue are assigned.
Track ticket	Current status is returned.

USER STORY 4 — Resident Lease & Policy AI Assistant

Owner: Member 4

User Story / Description

As a resident, I want to ask questions about my lease and building policies so that I can quickly find reliable information without manually searching documents.

Implementation Scope

- Create the React chat interface inside the same resident portal.
- Create a FastAPI endpoint for resident questions.
- Store lease-term and building-policy documents and embeddings using PostgreSQL + pgvector.
- Retrieve relevant document chunks using similarity search.
- Generate a grounded answer using the retrieved content.

Acceptance Criteria

- Resident can submit a question through the React chat.
- FastAPI searches the available lease and building-policy documents.
- The answer is generated from retrieved document content.
- Supported questions such as early termination or parking are answered from the documents.
- If information is unavailable, the assistant clearly says it cannot answer from the available documents.
- The assistant does not invent unsupported policy information.

Unit Test Cases

Test Case	Expected Result
Covered question	Relevant grounded answer is returned.
Vector search	Relevant document chunks are retrieved.
Unsupported question	Assistant says information is unavailable.
Empty question	API returns validation error.
No relevant chunks	No unsupported answer is generated.

HOW THE FOUR STORIES BECOME ONE APPLICATION

The team should integrate the stories into a single Resident Portal rather than creating four separate projects.

Resident Portal → Lease → Invoices → Maintenance → AI Assistant

Shared backend: FastAPI provides the APIs used by all React screens.

Shared database: PostgreSQL stores the application's transactional data including LeaseAgreement, RecurringInvoice and MaintenanceTicket. pgvector stores/searches embeddings for the RAG feature.

Integration: A resident can move between lease, invoice, maintenance and AI assistant sections without leaving the same application.

Team completion: Integrate and test all four stories, then complete the required UiPath monthly invoice automation, AWS/Azure deployment, CI/CD, monitoring/alerting and Jira Service Management incident triage before the final submission/demo.

Definition of Done: One working Meridian Residences application containing all four stories, integrated frontend, FastAPI backend/APIs, database, AI/RAG, maintenance agent, automation, unit tests and required production-readiness work.